

Operating Systems & Security Lab: The Quantum Widget Exploit

Duration: 3 Hours

Topic: Secure Bash Scripting, TOC-TOU Vulnerabilities, File Permissions, and Identity Management

Key Learning Objectives

By the end of this lab, students will be able to:

1. **Write Secure Bash Scripts:** Utilize variables, conditionals, loops, and functions while enforcing strict input validation and audit logging.
2. **Identify & Exploit Race Conditions:** Observe and document Time-of-Check to Time-of-Use (TOC-TOU) vulnerabilities caused by concurrent OS processes.
3. **Implement OS-Level Mutex:** Use file locking (flock) mechanisms to enforce mutual exclusion and patch concurrent execution flaws.
4. **Manage Linux Permissions:** Apply chmod to securely configure cross-user execution environments.
5. **Enforce Least Privilege (PoLP):** Design restricted drop-zone directories utilizing the sticky bit to protect shared assets without requiring root (sudo) privileges.

Mission Briefing (Incident Response)

You have just been hired as a DevSecOps Engineer for *QuantumTech*. The company is launching a highly anticipated, limited-edition product: the "Quantum Widget." We only have **100 widgets** in stock.

Threat Intelligence indicates that a cybercriminal syndicate has built a botnet to exploit our purchasing system the moment it goes live. They intend to abuse concurrent requests to drain our inventory, effectively stealing assets by paying for only a fraction of what they take.

Your job is to build the backend bash script, observe the exploit in real-time, patch the vulnerability, and secure the server environment for the rest of the team.

IMPORTANT SYSTEM NOTES: * **Environment:** For this lab, you must log in to the provided **shared Linux server** using your student account.

- **Privileges:** Your student account operates with **standard user privileges (NO sudo)**. You must design security controls that work within these constraints using clever permission structures.
- **Execution (PATH):** Your designated script folder is already included in your system's PATH environment variable. You can execute your scripts directly by name (e.g., `buy_widget`)

without needing the ./ prefix.

- **File Naming Convention:** All scripts must strictly follow standard Linux naming conventions: lowercase letters, words separated by underscores (snake_case), and **no file extensions** (do not use .sh).
- **Git Naming Convention:** You will push your final markdown report and observations to a Git repository. Your repository **MUST** be named using the following standard format: os-lab-quantum-<student_id> (e.g., os-lab-quantum-12345678).

Task Overview

Here is a quick summary of the 7 levels you will complete during this lab:

- **Level 1: Input Validation** - Create the buy_widget script with strict argument and integer validation.
- **Level 2: Audit Trails** - Add logging capabilities to buy_widget to track successful transactions.
- **Level 3: The Exploit (TOC-TOU)** - Create the bot_swarm script to simulate a concurrent attack and document the resulting race condition.
- **Level 4: The Patch (Mutex)** - Secure the buy_widget script using OS-level file locking (flock) to prevent the exploit.
- **Level 5: Red Team vs. Blue Team** - Pair up with a classmate, configure cross-user permissions, and test your mutex against their bot swarm.
- **Level 6: Secure Drop Zone (PoLP)** - Build the dropzone script to provision a secure folder where classmates can upload files but cannot delete yours.
- **Level 7: Forensic Cleanup** - Build the cleanup script to automatically organize generated log and dump files by their extension.

Level 1: Input Validation (Building the Engine)

First, we need a basic script to handle a single purchase request. In cybersecurity, **Rule #1 is: Never trust user input.**

1. Create a file named inventory.txt and simply type the number 100 inside it.
2. Create a new bash script named buy_widget and make it executable (chmod +x buy_widget).

Execute & Verify Tasks:

- **Special Variables:** The script must accept two arguments: the username (\$1) and the purchase_quantity (\$2).
- **Conditionals:** Check if the user actually provided exactly two arguments (\$#). If they didn't, print "Usage: buy_widget <username> <quantity>" and exit.
- **Sanitization:** Add an if statement to verify that \$2 (quantity) is a valid, positive integer. Without this, an attacker could input -50 to maliciously *increase* your inventory! Reject invalid transactions.

Level 2: Audit Trails (Functions & File I/O)

In incident response, if it isn't logged, it didn't happen. Let's add the actual business logic and an audit trail to `buy_widget`.

Execute & Verify Tasks:

1. **Create a Logging Function:** Write a function called `log_transaction()` that appends a record to a `sales.log` file.
 - *Anti-Cheat / Identity Requirement:* The log format MUST include your actual Student ID. Format: `[YourStudentID] [SUCCESS] <username> bought <quantity> widgets`.
2. **Read & Calculate:** * Read the current number from `inventory.txt` into a variable.
 - If the requested quantity is *less than or equal to* the current inventory, subtract the quantity and overwrite `inventory.txt` with the new number. Call `log_transaction`.
 - If they ask for more than is available, print "Transaction Failed: Not enough inventory!"

Observation Checkpoint 1:

Run the following commands exactly in order:

```
buy_widget Alice 5
```

```
buy_widget Hacker_Bob 200
```

```
buy_widget Eve -3
```

Verify: Check your `sales.log` and `inventory.txt`. Take a screenshot of the output to prove your input validation and audit logging work, and **embed it in your report.md file**.

Level 3: The Exploit (TOC-TOU Vulnerability)

Your script works perfectly for one user at a time. But what happens when 50 bots hit the system simultaneously? Let's simulate the attack.

1. Reset your `inventory.txt` to 100 and delete `sales.log`.
2. Create a new script called `bot_swarm` (`chmod +x bot_swarm`).
3. **Write the Exploit Loop:** Inside `bot_swarm`, write a for loop that runs 50 times.
4. Inside the loop, execute your `buy_widget` script, asking for 2 widgets each time. Add an ampersand (&) at the end of the command so they execute concurrently as background processes!

```
for i in {1..50}; do  
    buy_widget "Bot_$(i)" 2 &  
done  
wait
```

Observation Checkpoint 2 (Independent Work):

Run `bot_swarm`. Check `inventory.txt`. Is it 0?

You just experienced a **Time-of-Check to Time-of-Use (TOC-TOU) Vulnerability**, a classic form of a Race Condition. Multiple bots read `inventory.txt` (the check), did the math, and overwrote the file before the system could update (the use).

- **Task:** Reset inventory to 100, clear the log, and run the swarm script **5 separate times**.
- **Record:** In a text file named `observations.txt`, record the final number inside `inventory.txt` for each of the 5 runs. Write a 2-3 sentence explanation of *why* the number changes every single time based on OS process scheduling.

Level 4: The Patch (Mutex / File Locking)

To patch the TOC-TOU vulnerability, you must implement **Mutual Exclusion (Mutex)**. We must lock the `inventory.txt` file at the OS level so that only *one* process can interact with it at a time.

Execute & Verify Tasks:

Modify your `buy_widget` script to use the Linux `flock` command. Wrap your critical section inside a grouped command block tied to a file descriptor.

```
(  
  flock -x 200 # Acquire exclusive OS lock  
  # --- CRITICAL SECTION START ---  
  # Read inventory, do math, write inventory, log transaction  
  # --- CRITICAL SECTION END ---  
) 200> inventory.lock
```

Observation Checkpoint 3:

1. Reset `inventory.txt` to 100 and clear `sales.log`.
2. Run `bot_swarm` again.
3. **Verify:** Your `inventory.txt` MUST be exactly 0. **Take a screenshot showing the exact 0 inventory and the last 5 lines of your `sales.log`. Add this screenshot to your `report.md` file. The patch is successful.**

Level 5: Red Team vs. Blue Team (Cross-User Permissions)

Context: Your lock works against your own bots. But is it secure against external actors? You need to expose your API to a partner, but if you misconfigure Linux permissions (like using `chmod 777`), they could compromise your entire home directory.

Execute & Verify Tasks (Team Up!):

1. **Find a Partner:** Pair up with a classmate who is also logged into the shared server. Decide who is Player A and Player B.
2. **The DMZ (Public API):** Create a new folder named `public_api` in your home directory. Move your `buy_widget`, `inventory.txt`, `sales.log`, and `inventory.lock` into it.
3. **The Blue Team Puzzle:** Use the `chmod` command so your partner can run your script, but *nothing else*.
 - Your home directory `~` needs execute (`+x`) permissions so they can traverse into it.
 - Your `buy_widget` script needs read/execute permissions for others (`o+rx`).
 - Your `inventory.txt`, `sales.log`, and `inventory.lock` need read/write permissions for others (`o+rw`).
4. **The Red Team Attack:** Modify your `bot_swarm` script. Instead of running `buy_widget` from your own directory, change the path to point to your **partner's** absolute directory path (e.g., `/home/student_partner/public_api/buy_widget`). Have your partner do the same to you.
5. **Fire:** Both of you run your bot swarms at the *exact same time* against each other.

Observation Checkpoint 4:

Check your `sales.log`. Did your Mutex hold up against an external user?

Verify: **Take a screenshot of your `sales.log` proving the Red Team (your classmate) successfully executed your script without bypassing the lock. Embed this screenshot in your `report.md` file.**

Level 6: Principle of Least Privilege (The Secure Drop Zone)

Context: The CISO now wants external vendors (your classmates) to submit daily text reports to your server. We need a shared folder for them to upload files. Since you don't have `sudo` privileges to create isolated user accounts, you must use advanced permission controls to protect the shared directory.

Create a new script called `create_dropzone`. This script will automate the creation of a secure folder.

Execute & Verify Tasks:

1. **Interactive Prompt:** Use a `read` command to ask the operator what to name the drop zone folder (e.g., `vendor_reports`).
2. **Directory Setup:** The script should create this directory in your home folder.
3. **The Sticky Bit Puzzle:** You need to allow *anyone* to write/upload to this folder. Use `chmod` to grant write execute permissions to others (`o+wx`).
 - **Crucially**, you must also apply the **sticky bit** (`chmod +t <directory>`). The sticky bit

ensures that even in a highly permissive folder, users can only delete or modify files *they personally own*.

Observation Checkpoint 5 (Test the Drop Zone):

1. Run your script to create the directory. Create a file inside it called my_rules.txt.
2. Ask your partner to navigate into your drop zone and create a file called partner_report.txt.
3. Ask your partner to try and delete *your* my_rules.txt file. They should receive a "Permission denied" error.

Verify: **Take a screenshot of the ls -ld output showing the t in your drop zone's permissions (e.g., drwxrwxrwt). Add it to your report.md file.**

Level 7: Forensic Cleanup (Log Management)

Context: The Red Team bot swarm generated hundreds of temporary error files, log files, and receipt files. A SOC analyst needs these sorted by file extension to properly parse them for indicators of compromise (IoCs).

Create a script called cleanup to organize the forensic data.

Execute & Verify Tasks:

1. **Setup:** Manually create 3 dummy files: test.log, data.csv, error.tmp.
2. **The Script:** Write a script that loops through all files in the current directory.
3. **Identify & Move:** For each file, identify its extension (e.g., .log, .csv). Check if a folder for that extension exists. If not, create it. Then, move the file into its respective folder.

Verify: **Run tree or ls -R to show that the files have been properly sorted into log/, csv/, and tmp/ directories for the forensic team. Take a screenshot of the output and add it to your report.md file.**

Final Deliverables

Note: You do **not** need to submit your actual script files. Your instructor will review and grade your scripts (buy_widget, bot_swarm, create_dropzone, cleanup) directly on the shared Linux server. Please ensure they are saved with the correct names and are fully executable in your designated directory.

Push the following files to your assigned Git repository. **Your repository must be named according to the standard convention: os-lab-quantum-<student_id>.**

1. Your observations.txt file detailing your TOC-TOU Race Condition experiment.
2. A **Markdown file** (e.g., report.md) containing your embedded **Verification Screenshots** from Levels 2, 3, 5 (Red Team log), 6 (Sticky bit ls -ld output), and 7. (Note: Make sure to push the actual image files to your repository so the markdown links render correctly!)